

EmbedSim Framework

Fachschale Tutorial

Buck Converter — PI Voltage Controller

A complete guide: Modelica plant → C algorithm → Cython bridge → Python block → CodeGen → Plots

EmbedSim v3.1.0 | March 2026



https://github.com/vectorsim/embed_sim_project

1. What Is a Fachschale?

The German term Fachschale ("domain shell") describes a self-contained module that bundles all layers needed for one embedded control function. In EmbedSim a Fachschale always consists of four layers:

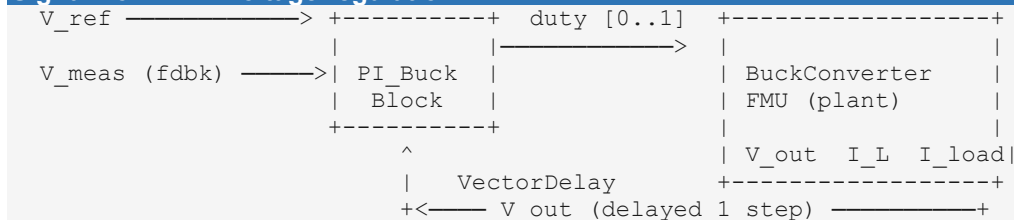
Layer	Purpose
Plant model (.mo / .fmu)	Physics — Modelica ODE compiled to FMI 2.0 FMU. Simulates the system under control.
C controller (.c / .h)	Algorithm — portable MISRA-C, zero OS dependency. Identical binary on Aurix TriCore and ARM Cortex-M4.
Cython wrapper (.pyx)	Bridge — exposes the C struct to Python as a cdef class. Float32, near-zero overhead, no GIL.
Python block (.py)	Orchestration — derives from SimBlockBase. Plugs into VectorSim, drives RK4, carries CodeGen metadata.

The critical benefit: the C controller is never rewritten. The same source file runs in simulation via Cython and is compiled with TASKING ctc and flashed to the Aurix ECU.

2. System Overview

2.1 Closed-Loop Signal Flow

Signal flow — PI voltage regulation



The one-step VectorDelay on the feedback path is mandatory. It breaks the algebraic loop so EmbedSim's DFS topological scheduler can assign a strict execution order without iteration.

2.2 File Map

File	Location	Role
BuckConverter.mo	buck_converter/modelica/	Modelica plant ODE
BuckConverter.fmu	buck_converter/modelica/	Compiled FMI 2.0 unit
BuckConverterBlock.py	buck_converter/	Auto-generated FMUBlock class
pi_buck_controller.c/h	electrical_blocks/c_src/	Portable MISRA-C PI algorithm
pi_buck_wrapper.pyx	electrical_blocks/c_src/	Cython bridge (float32)
setup_pi_buck.py	electrical_blocks/c_src/	Cython build script

File	Location	Role
build_pi_buck.bat	electrical_blocks/c_src/	Compile and copy .pyd
pi_buck_block.py	buck_converter/	VectorBlock (RK4 + CodeGen)
pi_buck_example.py	examples/pi_buck_converter/	Runnable simulation demo

3. Plant Model (BuckConverter.mo)

The plant is the physical system under control. In EmbedSim it is always modelled in OpenModelica and compiled to an FMU. The auto-generator `mo_to_fmu_client.py` then writes a typed FMUBlock Python class from the parsed `.mo` source.

3.1 Modelica Equations

BuckConverter.mo

```

model BuckConverter
  "Simple buck converter plant model"
  parameter Real L      = 1e-4  "Inductance [H]";
  parameter Real C      = 1e-4  "Capacitance [F]";
  parameter Real R_load = 10.0   "Load resistance [Ohm]";
  parameter Real V_in   = 24.0   "Input voltage [V]";
  parameter Real f_sw   = 1e5    "Switching frequency [Hz]";
  input  Real duty      "PWM duty cycle [0-1]";
  output Real V_out     "Output voltage [V]";
  output Real I_L       "Inductor current [A]";
  output Real I_load    "Load current [A]";
  Real V_sw;
equation
  V_sw = duty * V_in;
  L * der(I_L) = V_sw - V_out; // inductor ODE
  C * der(V_out) = I_L - I_load; // capacitor ODE
  I_load = V_out / R_load; // resistive load
end BuckConverter;

```

Steady-state operating point: $V_{out} = \text{duty} \times V_{in}$. For $V_{out} = 12\text{ V}$, $V_{in} = 24\text{ V}$ the duty converges to 0.5.

3.2 Generating the FMU Block (gen_fmu.py)

gen_fmu.py

```

from mo_to_fmu_client import generate_fmu_block

mo_path      = r'C:\EmbedSimProject\buck_converter\modelica\BuckConverter.mo'
output_dir   = r'C:\EmbedSimProject\buck_converter'

output_file = generate_fmu_block(mo_path, output_dir, verbose=True)
# Writes: buck_converter/BuckConverterBlock.py

```

NOTE

*Re-run `gen_fmu.py` whenever `BuckConverter.mo` changes. The generated file carries a **DO NOT EDIT** header — always regenerate rather than edit manually.*

4. C Controller (pi_buck_controller.c)

The PI algorithm is written in portable C with no platform dependencies. It targets Infineon Aurix TriCore as primary and ARM Cortex-M4 as secondary. The same source file runs unchanged in simulation via the Cython wrapper.

4.1 Algorithm

PI with anti-windup — discrete recurrence

```
e(k)      = V_ref(k) - V_meas(k)
integ(k)  = clamp( integ(k-1) + e(k)*dt,
                  -duty_max/Ki, +duty_max/Ki ) // anti-windup
duty(k)   = clamp( Kp*e(k) + Ki*integ(k),
                  duty_min, duty_max )
```

The integrator clamp is the anti-windup mechanism: it prevents $K_i \times \text{integ}$ from exceeding duty_max regardless of how long the error persists.

4.2 Data Structures

pi_buck_controller.h — structs

```
typedef struct {
    real32_T Kp;          // Proportional gain [1/V]
    real32_T Ki;          // Integral gain [1/(V*s)]
    real32_T duty_max;    // Max duty cycle [0-1]
    real32_T duty_min;    // Min duty cycle [0-1]
    real32_T Ts;          // Nominal sample time [s]
} PI_Buck_Params_T;

typedef struct {
    real32_T integrator;  // Accumulated error integral
    real32_T prev_error;  // Previous error (diagnostics)
    real32_T last_output; // Last duty cycle
} PI_Buck_State_T;

typedef struct {          // One instance per channel
    PI_Buck_Params_T params;
    PI_Buck_State_T state;
} PI_Buck_Block_T;

typedef struct { real32_T V_ref; real32_T V_meas; } PI_Buck_Input_T;
typedef struct { real32_T duty; } PI_Buck_Output_T;
```

WHY real32_T?

real32_T (= float) comes from Sys_Types.h, the EmbedSim standard header adopted from Simulink Coder ERT for Aurix TriCore. Using it throughout guarantees bit-identical float32 arithmetic between simulation and ECU firmware.

4.3 API

pi_buck_controller.h — function prototypes

```
void PI_Buck_Init      (PI_Buck_Block_T* pPI);
void PI_Buck_SetParams (PI_Buck_Block_T* pPI,
                        real32_T Kp, real32_T Ki,
                        real32_T duty_max, real32_T duty_min,
                        real32_T Ts);
void PI_Buck_ResetState(PI_Buck_Block_T* pPI);
void PI_Buck_Compute   (PI_Buck_Block_T* pPI,
                        const PI_Buck_Input_T* pIn,
                        real32_T dt,
                        PI_Buck_Output_T* pOut);
```

5. Cython Wrapper (pi_buck_wrapper.pyx)

The Cython wrapper bridges C and Python. PI_Buck_Block_T is stored by value inside the cdef class — no heap allocation, no pointer crossing the Python/C boundary.

5.1 Wrapper Class

pi_buck_wrapper.pyx

```
# distutils: sources = pi_buck_controller.c
# cython: language_level = 3

cdef class PI_BuckWrapper:
    cdef PI_Buck_Block_T _block    # stored by value

    def __cinit__(self): PI_Buck_Init(&self._block)

    def set_params(self, float Kp, float Ki,
                  float duty_max, float duty_min, float Ts):
        PI_Buck_SetParams(&self._block, Kp, Ki, duty_max, duty_min, Ts)

    def compute(self, float V_ref, float V_meas, float dt) -> float:
        cdef PI_Buck_Input_T inp
        cdef PI_Buck_Output_T out
        inp.V_ref = V_ref
        inp.V_meas = V_meas
        PI_Buck_Compute(&self._block, &inp, dt, &out)
        return out.duty

    # RK4 state synchronisation
    def set_integrator(self, float v): self._block.state.integrator = v
    def get_integrator(self) -> float: return self._block.state.integrator
    def get_last_output(self) -> float: return self._block.state.last_output
    def reset(self): PI_Buck_ResetState(&self._block)
```

5.2 Compiling

Build commands

```
cd C:\EmbedSimProject\electrical_blocks\c_src

REM Option A: compile + auto-copy .pyd
build_pi_buck.bat

REM Option B: manual
python setup_pi_buck.py build_ext --inplace
```

COMPILE ONCE

After a successful build the .pyd is a self-contained binary. Recompile only when pi_buck_controller.c or pi_buck_wrapper.pyx changes.

6. Python Block (pi_buck_block.py)

PI_BuckBlock derives from SimBlockBase and handles three concerns: RK4 state integration, dual Python/C backend dispatch, and CodeGen metadata for embedsim_loop.c emission.

6.1 CodeGen Attributes

PYXInspector reads these class-level attributes when generating embedsim_loop.c:

pi_buck_block.py — CodeGen metadata

```
class PI_BuckBlock(SimBlockBase):
    PYX_FILE      = '../c_src/pi_buck_wrapper.pyx' # absolute path
    step_func     = 'PI_Buck_Compute'             # C function name
    state_struct  = 'PI_Buck_Block_T'             # C state struct
    NUM_INPUTS    = 2                             # V_ref, V_meas
    OUTPUT_SIZE   = 1                             # duty
    C_SOURCES     = ['pi_buck_controller.c']
    C_HEADERS     = ['pi_buck_controller.h', 'Sys_Types.h']
```

6.2 RK4 State Integration

get_derivative() — RK4 interface

```
self.state = np.zeros(1, dtype=np.float32) # state[0] = integrator

def get_derivative(self, t, input_values=None):
    """dx/dt = error - RK4 integrates this to produce integ(t)"""
    if t < self.t_enable: # soft-start guard
        return np.zeros(1, dtype=np.float32)
    V_ref, V_meas = self._get_voltages(input_values)
    return np.array([V_ref - V_meas], dtype=np.float32)
```

6.3 Dual Backend Dispatch

Backend selection and C↔Python state sync

```
# Python backend — no compilation needed
pi = PI_BuckBlock('pi', Kp=0.15, Ki=8.0, use_c_backend=False)

# C backend — requires compiled pi_buck_wrapper.pyd
pi = PI_BuckBlock('pi', Kp=0.15, Ki=8.0, use_c_backend=True)

def compute_c(self, t, dt, input_values):
    self._impl.set_integrator(np.float32(self.state[0])) # Python→C
    duty = self._impl.compute(V_ref, V_meas, dt)
    self.state[0] = np.float32(self._impl.get_integrator()) # C→Python
```

7. Code Generator (code_generator.py)

The EmbedSim code generator emits `embedsim_loop.c` and `embedsim_loop.h` into `embedsim_gen/` at project root. These files are the MCU deployment artefact — a self-contained C loop compilable with TASKING ctc for Aurix TriCore.

7.1 Architecture

Class	File	Responsibility
SimBlockBase	embedsim/code_generator.py	Base class; carries PYX_FILE, step_func, C_SOURCES, C_HEADERS
PYXInspector	electrical_blocks/pyx_inspector.py	Parses .pyx; extracts C function signatures and port counts
CodeGenStart	embedsim/code_generator.py	Transparent passthrough; marks INPUT boundary of generated region. Wired inline with >>.
CodeGenEnd	embedsim/code_generator.py	Transparent passthrough; marks OUTPUT boundary of generated region. Wired inline with >>.

Class	File	Responsibility
LoopGenerator	embedsim/code_generator.py	Walks sorted block list between CodeGenStart and CodeGenEnd; calls <code>_emit_block()</code> for each SimBlockBase; writes <code>embedsim_loop.c / .h</code>
<code>_emit_block()</code>	embedsim/code_generator.py	Emits one C function call with explicit signal wiring
<code>_gen_h()</code>	embedsim/code_generator.py	Emits <code>#include</code> directives and extern register declarations

7.2 Triggering Code Generation

CodeGenStart and CodeGenEnd are transparent VectorBlock passthroughs. They must be wired inline into the signal graph with the `>>` operator, exactly like any other block. After `sim.run()` completes, call `LoopGenerator.generate()` to emit the files.

Correct CodeGen pattern — inline wiring + LoopGenerator

```
from embedsim.code_generator import CodeGenStart, CodeGenEnd, LoopGenerator

# — Step 1: create boundary markers —————
cg_start = CodeGenStart('pi_ctrl_start')
cg_end   = CodeGenEnd('pi_ctrl_end')

# — Step 2: wire inline into the graph —————
v_ref >> cg_start >> pi_controller >> cg_end >> buck_plant >> sink

# — Step 3: run simulation (boundary markers are pass-through) —
sim.run(verbose=True)

# — Step 4: emit C files —————
gen = LoopGenerator(cg_start, cg_end)
gen.generate(output_dir='C:\\EmbedSimProject\\embedsim_gen', dt_hz=1e6)
# Writes:
#   embedsim_gen/embedsim_loop.c
#   embedsim_gen/embedsim_loop.h
```

IMPORTANT

CodeGenStart and CodeGenEnd are NOT context managers. Do not use: with CodeGenStart(): `sim.run()` — this will fail. They must be wired with `>>` before sim is created, and LoopGenerator called after `sim.run()` completes.

7.3 Generated C Loop

embedsim_loop.c (representative output)

```
/* embedsim_loop.c
 * Auto-generated by EmbedSim LoopGenerator (feature 05121967)
 * DO NOT EDIT — re-generate with LoopGenerator.generate()
 */
#include <string.h> /* memcpy */
#include "embedsim_loop.h"
#include "pi_buck_controller.h"

void embedsim_loop_step(real32_T dt)
{
    /* Block: pi_buck (PI_BuckBlock) */
    real32_T u_pi_buck[2];
```

```

u_pi_buck[0] = y_vref[0];
u_pi_buck[1] = y_fb_delay[0];
real32_T y_pi_buck[1];
PI_Buck_Compute(&pi_buck_state, u_pi_buck, dt, y_pi_buck);

/* FMU blocks omitted on MCU target */
}

```

**MCU
NOTE**

FMU blocks (OpenModelica plant models) are simulation-only and are omitted from embedsim_loop.c. Only SimBlockBase-derived blocks with C_SOURCES / C_HEADERS are emitted. The LoopGenerator DFS traversal visits only blocks inside the cg_start → cg_end boundary.

7.4 Generated Header

embedsim_loop.h

```

/* embedsim_loop.h
 * Auto-generated by EmbedSim LoopGenerator (feature 05121967)
 */
#ifndef EMBEDSIM_LOOP_H
#define EMBEDSIM_LOOP_H

#include "Sys_Types.h" /* real32_T */

/* Sample period for a 1000000 Hz control loop */
#define EMBEDSIM_DT (0.0000010000f)

#include "pi_buck_controller.h"

/* Persistent state (file-scope statics — one per stateful block) */
static PI_Buck_Block_T pi_buck_state;

void embedsim_loop_init(void);
void embedsim_loop_step(real32_T dt);

#endif /* EMBEDSIM_LOOP_H */

```

7.5 Common CodeGen Pitfalls

Error / Symptom	Root Cause	Fix
ImportError: cannot import name 'CodeGenRegion'	CodeGenRegion was an early prototype name, removed in v3.	Import CodeGenStart, CodeGenEnd, LoopGenerator instead.
3 block(s) in region instead of 1	cg_start and cg_end passthrough blocks are counted by the DFS traversal.	Expected behaviour — passthrough blocks count. Only PI_BuckBlock emits a real C call.
embedsim_loop.c contains only /* [pass-through] */ comments	Block is missing PYX_FILE / step_func class attributes, or does not derive from SimBlockBase.	Ensure the block class declares step_func, state_struct, C_SOURCES, C_HEADERS.
UnicodeEncodeError on Windows when writing .c / .h	write_text() defaulted to cp1252 instead of UTF-8.	Fixed in code_generator.py v1.6: all write_text() calls use encoding='utf-8'.

Error / Symptom	Root Cause	Fix
AttributeError: 'VectorScope' has no attribute 'time'	VectorScope stores time in .t not .time.	Use scope.t (list) or scope._buf_t[:scope._step_idx] (pre-allocated path).
scope.data keys not found	Multi-index signals are stored as 'label[0]', 'label[1]', not plain 'label'.	Look up f"{label}[{index}]" first, fall back to plain label for scalars.

8. Running the Simulation (pi_buck_example.py)

pi_buck_example.py assembles all blocks into a closed-loop VectorSim graph, prints the topology, runs the simulation, triggers CodeGen, and plots the results.

8.1 Imports and Block Construction

§1 — Block construction

```

from embedsim.simulation_engine import EmbedSim, ODESolver, VectorDelay
from embedsim.source_blocks import VectorStep
from embedsim.dynamic_blocks import VectorEnd
from embedsim.code_generator import CodeGenStart, CodeGenEnd, LoopGenerator
from embedsim.topology_printer import TopologyPrinter

v_ref = VectorStep('vref', step_time=0.001,
                  before_value=0.0, after_value=12.0, dim=1)
pi_controller = PI_BuckBlock(name='pi_buck', Kp=0.15, Ki=8.0,
                             duty_max=0.9, duty_min=0.1, Ts=1e-4,
                             use_c_backend=True)
buck_plant = BuckConverterBlock(name='buck', fmu_path=...,
                                L=100e-6, C=100e-6, R_load=10,
                                V_in=24, f_sw=100e3)
feedback_delay = VectorDelay('fb_delay', initial=[0.0])
sink = VectorEnd('sink')
cg_start = CodeGenStart('pi_ctrl_start')
cg_end = CodeGenEnd('pi_ctrl_end')

```

8.2 Wiring

§2 — Wiring

```

# Forward path with CodeGen boundary markers inline
v_ref      >> cg_start      # V_ref enters codegen region
cg_start   >> pi_controller  # V_ref → PI port 0
pi_controller >> cg_end      # duty exits codegen region
cg_end     >> buck_plant    # duty → FMU input
buck_plant >> sink

# Feedback path
buck_plant >> feedback_delay # V_out → delay
feedback_delay >> pi_controller # delayed V_meas → PI port 1

```

8.3 Topology Printer

§3 — Topology printer

```

sim = EmbedSim(sinks=[sink], T=0.01, dt=1e-6, solver=ODESolver.RK4)
printer = TopologyPrinter(sim, title='Buck Converter - PI Voltage Control')
printer.print_console() # ASCII diagram to stdout
printer.show_gui()      # GUI window (close to continue)

```

NOTE

printer.show_gui() opens a GUI block-diagram window before the run starts. Close the window to allow execution to continue to sim.run().

8.4 Simulation and CodeGen

§4 — Simulation + CodeGen

```
sim.scope.add(v_ref, label='v_ref')
sim.scope.add(pi_controller, label='pi_ctrl')
sim.scope.add(buck_plant, label='buck_out', indices=[0, 1])
sim.scope.add(feedback_delay, label='fb_delay')

sim.run(verbose=True, progress_bar=True)

# Emit C files after the run
gen = LoopGenerator(cg_start, cg_end)
gen.generate(output_dir=codegen_dir, dt_hz=1e6)
# → embedsim_gen/embedsim_loop.c
# → embedsim_gen/embedsim_loop.h
```

8.5 Scope Data Access (PlotHelper)

VectorScope stores recorded data in two possible forms depending on whether the pre-allocation path was used. PlotHelper handles both transparently:

Scope data access pattern

```
scope = sim.scope

# Time — list path (small sims) or pre-allocated ndarray (large sims)
t_raw = scope.t if scope.t else scope._buf_t[:scope._step_idx]
t_ms = np.asarray(t_raw) * 1e3 # convert to milliseconds

# Data keys — multi-index signals stored as 'label[0]', 'label[1]'
key = f"{label}[{index}]" # e.g. 'buck_out[0]' for V_out
if key not in scope.data and label in scope.data:
    y = np.array([float(v) for v in scope.data[label]]) # scalar
else:
    y = np.array([float(v) for v in scope.data[key]]) # indexed
```

8.6 Simulation Results

Response to a 12 V step at $t = 1$ ms. Parameters: $K_p = 0.15$, $K_i = 8.0$, $L = 100 \mu\text{H}$, $C = 100 \mu\text{F}$, $R_{\text{load}} = 10 \Omega$, $V_{\text{in}} = 24$ V.

Observations:

- V_{out} rises from 0 V and tracks toward 12 V. Overshoot (~ 16 V) results from the aggressive $K_i = 8.0$ setting.
- Duty cycle saturates at 90 % (duty_max) during the transient, then settles near 45 % — consistent with the ideal $12/24 = 0.5$ operating point.
- Residual oscillation at 10 ms indicates the gains need further reduction. See Section 9.
- Inductor current I_L mirrors V_{out} and stabilises near 1.2 A ($= 12 \text{ V} / 10 \Omega$).

9. Tuning Guide

The five parameters below are the primary tuning knobs. Changes take effect via `set_params()` — no recompilation needed.

9.1 Parameter Reference

Parameter	Default	Effect	Adjust if...
Kp	0.15	Proportional gain: immediate duty correction per volt of error.	Overshoot too large → decrease. Sluggish rise → increase.
Ki	8.0	Integral gain: eliminates steady-state offset over time.	Residual offset → increase. Growing oscillation → decrease.
duty_max	0.90	Hard ceiling on duty cycle. Protects inductor from saturation.	Inductor saturates at startup → reduce to 0.75–0.80.
duty_min	0.10	Hard floor. Avoids discontinuous conduction mode at light load.	Output collapses at light load → try 0.05.
Ts	1e-4	Nominal sample time. Used when <code>dt = 0</code> is passed to <code>Compute()</code> .	Mismatch with simulation <code>dt</code> causes integration drift.

9.2 Systematic Tuning Sequence

1. Set `Ki = 0`. Increase `Kp` until `V_out` rises fast with acceptable overshoot (< 15 %).
2. Freeze `Kp`. Increase `Ki` from 0 until steady-state offset disappears within 3–5 ms.
3. If oscillation grows, halve `Ki`. If offset remains, double `Ki`.
4. Tighten `duty_max` to the largest duty needed at steady state plus 10 % headroom.
5. Verify both backends (`use_c_backend=True` and `False`) give the same steady-state voltage.

9.3 Runtime Update

Runtime tuning — no restart needed

```
pi_controller.set_params(Kp=0.08, Ki=4.0) # reduced gains
sim.reset()
sim.run()

pi_controller.set_params(duty_max=0.80) # tighter ceiling
```

10. Creating a New Fachschale

The Buck Converter pattern generalises to any controller. Follow this checklist:

6. Write the Modelica plant (.mo) with explicit input/output/parameter declarations. Compile to FMU. Run `gen_fmu.py`.
7. Write the controller in plain C using `real32_T`. Structure it as `Init` / `SetParams` / `Reset` / `Compute` with anti-windup on all integrator paths.
8. Write the Cython .pyx wrapper as a `cdef` class storing the C struct by value. Add `set_integrator` / `get_integrator` for every continuous state.
9. Write the Python block deriving from `SimBlockBase`. Populate `PYX_FILE`, `step_func`, `state_struct`, `C_SOURCES` and `C_HEADERS`. Implement `get_derivative()` for each continuous state.
10. Add `setup_xxx.py` and `build_xxx.bat` following the `pi_buck` pattern. Run `build_xxx.bat`.

11. Wire a simulation graph and verify steady-state against the analytical operating point.
12. Wire CodeGenStart and CodeGenEnd around the target blocks (Section 7.2) and call LoopGenerator.generate(). Verify embedsim_loop.c compiles clean with TASKING etc.

11. Quick-Reference Card

End-to-end command sequence from Modelica source to MCU-ready C output:

Full workflow — project root

```
# 1. Generate FMUBlock from Modelica source
python gen_fmu.py

# 2. Compile Cython extension
cd electrical_blocks\c_src
build_pi_buck.bat
cd ..\..\

# 3. Run simulation + CodeGen (all-in-one)
cd examples\pi_buck_converter
python pi_buck_example.py
# Output: embedsim_gen/embedsim_loop.c
#         embedsim_gen/embedsim_loop.h

# 4. Build all extensions from project root
cd C:\EmbedSimProject && build_all.bat
```

★ NEXT

The PMSM FOC example in examples/pmsm/ applies the same pattern to a full FOC chain: Clarke/Park, SVPWM, SMC current controller, SMO observer, and Speed PI — all wired with >> under one RK4 engine.